

BCC2001

Algoritmos



Diego Bertolini
diegobertolini@gmail.com

Aula

- Estrutura de Dados Composta
 - Matrizes em algoritmos.
 - Matrizes em C.

Estrutura de dados composta

- São um conjunto de variáveis identificadas por um mesmo nome.
- **Homogêneas**
 - Arranjos unidimensionais → vetores
 - Arranjos bidimensionais → matrizes
 - Multidimensionais
- **Heterogêneas**
 - Estruturas → structs → registros

Agregados Homogêneos Multidimensionais.

- Agregado Homogêneo Multidimensional
 - **Sinônimos:**
 - Matriz (bidimensional)
 - Conjunto multidimensional
 - Vetor multidimensional
 - Array multidimensional
- Um conjunto de dados pode ter mais de uma dimensão;
- É um agregado homogêneo de dados estruturado em mais de uma dimensão. O mais utilizado é o agregado bidimensional (aquele que possui duas dimensões);

Agregados Homogêneos Multidimensionais.

- Utilizados para armazenar conjuntos de dados cujos elementos necessitam ser endereçados por mais de um índice.
- Também são conhecidos como arrays ou matrizes.
- Como declarar:

```
<tipo> nome [tamanho1][tamanho2]...;
```

Agregados Homogêneos Multidimensionais.

- De 2 dimensões (matriz):
- Declaração:

```
<tipo> nome[<tam1>][<tam2>];
```

- **Visualização:**

```
int matriz [m][n];
```

	0	1	2	...	n-1
0	788	598	265	...	156
1	145	258	369	...	196
2	989	565	345	...	526
⋮	⋮	⋮	⋮	⋱	⋮
m-1	845	153	564	892	210

Agregados Homogêneos Multidimensionais.

- De 3 dimensões (matriz):
- Declaração:

```
<tipo> nome[<tam1>][<tam2>][<tam3>];
```

- **Visualização:**

```
int matriz [x][y][z];
```


Agregados Homogêneos Multidimensionais.

- **RELEMBRANDO:**

- O tamanho de um vetor ou matriz é pré-definido, ou seja, após a compilação, não pode ser alterado.
- A declaração e a alocação é estática.
 - Mantém o mesmo tamanho ao longo de toda a execução do programa.

Formas de Inicialização

- Atribuir valores na própria declaração do vetor:

```
int matriz[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int matriz[3][3], i, j;
7
8     for (i = 0; i < 3; i++){
9         for (j = 0; j < 3; j++){
10             matriz[i][j] = 0;
11             printf("Matriz[%d][%d] --> %d\n", i,j, matriz[i][j] );
12         }
13     }
14
15     printf("\n\n");
16     system("PAUSE");
17 }
```

Formas de Inicialização

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define MAX_LINE 3
4 #define MAX_COL 3
5
6 int main()
7 {
8     int matriz[MAX_LINE][MAX_COL], i, j;
9
10    for (i = 0; i < MAX_LINE; i++){
11        for (j = 0; j < MAX_COL; j++){
12            printf("Matriz[%d][%d] -->: ", i, j);
13            scanf("%d", &matriz[i][j]);
14        }
15    }
16    printf("\n\n");
17    //Imprimindo os Valores
18    for (i = 0; i < MAX_LINE; i++){
19        for (j = 0; j < MAX_COL; j++){
20            printf("[%d] ", matriz[i][j]);
21        }
22        printf("\n");
23    }
24
25    printf("\n\n");
26    system("PAUSE");
27 }
```

```
Matriz[0][0] -->: 10
Matriz[0][1] -->: 20
Matriz[0][2] -->: 30
Matriz[1][0] -->: 40
Matriz[1][1] -->: 50
Matriz[1][2] -->: 60
Matriz[2][0] -->: 70
Matriz[2][1] -->: 80
Matriz[2][2] -->: 90
```

```
[10] [20] [30]
[40] [50] [60]
[70] [80] [90]
```

BOA PRÁTICA

- Definir constantes para o tamanho de vetores e matrizes ;
 - Deixa o código flexível e facilita a manutenção.

```
#define MAX_DIM 10
```

```
// Declara um vetor de MAX_DIM elementos.  
// Alocação estática.
```

```
int vetorA[MAX_DIM];
```

```
int vetorB[MAX_DIM];
```

```
int vetorC[MAX_DIM];
```

```
unsigned int 1;
```

Definindo
Constantes

Usando as
Constantes

```
#define MAX_DIM_LIN 3
```

```
#define MAX_DIM_COL 3
```

```
int A[MAX_DIM_LIN][MAX_DIM_COL];
```

```
int B[MAX_DIM_LIN][MAX_DIM_COL];
```

```
int A_X_B[MAX_DIM_LIN][MAX_DIM_COL];
```

Exercício

Determinar:

1. $M[3][0]$
2. $M[4][2]$
3. $M[1][3]$
4. $M[5][M[0][2]]$
5. $M[M[3][1]][1]$
6. $M[4][(M[1][2] + M[3][0])]$

	0	1	2	3
0	1	2	3	4
1	5	-5	3	0
2	1	1	1	1
3	-3	2	0	0
4	0	0	1	1
5	-1	-1	-2	-2

Exercício

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define MAX_LINE 6
4 #define MAX_COL 4
5
6 int main()
7 {
8     int matriz[MAX_LINE][MAX_COL], i, j;
9
10    for (i = 0; i < MAX_LINE; i++){
11        for (j = 0; j < MAX_COL; j++){
12            printf("Matriz[%d][%d] -->: ", i, j);
13            scanf("%d", &matriz[i][j]);
14        }
15    }
16    printf("\n\n");
17    //Imprimindo os Valores
18    printf("Matriz[3][0]: %d ", matriz[3][0]);
19    printf("\nMatriz[4][2]: %d", matriz[4][2]);
20    printf("\nMatriz[1][3]: %d", matriz[1][3]);
21    printf("\nMatriz[5][0 2]: %d", matriz[5][matriz[0][2]]);
22    printf("\nMatriz matriz[matriz[5][1]][1]: %d", matriz[matriz[5][1]][1]);
23    printf("\nMatriz matriz[4][(matriz[1][2] + matriz[3][0])]: %d", matriz[4][(matriz[1][2] + matriz[3][0])]);
24    printf("\n\n");
25    system("PAUSE");
26 }
```

```
Matriz[3][0]: -3
Matriz[4][2]: 1
Matriz[1][3]: 0
Matriz[5][0 2]: -2
Matriz matriz[matriz[5][1]][1]: 0
Matriz matriz[4][(matriz[1][2] + matriz[3][0])]: 0
```

Exercício

- 1) Faça um programa que leia uma matriz 3x4 de inteiros, calcule e imprima a soma de todos os seus elementos.
- 1) Faça um programa que leia duas matrizes 3x3 de inteiros, gere e imprima a matriz diferença.

Exercícios...

Lista Vetor / Matriz.

Dúvidas, Críticas ou Sugestões

diegobertolini@gmail.com